

Reto: Sistema completo, documentación y presentación final

Sonrisas de Cristal

Instituto Tecnológico y de Estudios Superiores de Monterrey
Escuela de Ingeniería y Ciencias
Campus Monterrey

Integración de seguridad informática en redes y sistemas de software
(Gpo 403)

Equipo: NeuralHub

Socio formador: Fundación Estancias Infantiles Sonrisas de Cristal

Estudiantes

Santiago Hernández Senosiain	A01787550
Gilberto Olavarrieta Sánchez	A01286889
Franco Salazar Guerrero	A01178655
Gerardo Islas Gómez	A01739896

Profesores

Juan David Guarnizo Hernández
Rolando Evelio Pérez Versón
Aída Judith Gándara Tovar

Monterrey, Nuevo León, 22 de octubre de 2026

Índice

1. Resumen del alcance final **1**
1.1. Problema . . . 1
1.2. Alcance entregado . . . 1
1.3. Referencia al acta de acuerdos . . . 2

2. Arquitectura final del sistema **2**
2.1. Capas y componentes . . . 2
2.2. Decisiones técnicas principales . . . 3

3. Matriz de trazabilidad y evidencia de pruebas **4**
3.1. Niveles de prueba . . . 4
3.2. Resultados . . . 5
3.3. Matriz de trazabilidad . . . 5
3.4. Evidencia . . . 8

4. Evidencia de los controles de seguridad **9**
4.1. Controles en la API . . . 9
4.2. Infraestructura y despliegue seguro . . . 11
4.3. Auditoría OWASP . . . 11
4.4. Escaneo dinámico con OWASP ZAP . . . 12

5. Manual de usuario **14**
5.1. Entrar a la app . . . 14
5.2. Educadora . . . 14
5.3. Familia . . . 15
5.4. Dirección . . . 15
5.5. Fundación . . . 15

6. Manual de instalación y ejecución **15**
6.1. Requisitos . . . 16
6.2. Levantar el backend (un comando) . . . 16
6.3. Instalar y conectar la app . . . 16
6.4. Operación . . . 17
6.5. Correr las pruebas . . . 17
6.6. Compilar el APK de release firmado . . . 18

7. Plan de implantación **18**

8. Administración final del proyecto **20**
8.1. Backlog final . . . 20
8.2. Estado de los requerimientos . . . 21
8.3. Bitácora de hitos . . . 21
8.4. Desviaciones . . . 22
8.5. Riesgos . . . 22
8.6. Distribución del trabajo . . . 22
8.7. Retrospectiva . . . 23

9. Consideraciones éticas y legales **24**
9.1. Criterios éticos y legales convertidos en requisitos verificables 24
9.2. Aviso de privacidad 25
9.3. Clasificación de los datos 28
9.4. Dilemas éticos y decisiones de diseño 31

10. Entregables y enlaces **33**

A. Archivos de evidencia **33**

1. Resumen del alcance final

1.1. Problema

Las estancias de la Fundación Estancias Infantiles Sonrisas de Cristal llevaban la bitácora diaria en papel. Las educadoras tomaban fotos de los niños con su celular personal y las enviaban por WhatsApp, así que las imágenes de menores quedaban en teléfonos particulares, sin control. Además, cada estancia pagaba su propia plataforma (cerca de $2,900 al mes) y la comunicación con las familias estaba dispersa.

1.2. Alcance entregado

Sonrisas de Cristal es una sola plataforma para toda la Fundación, separada por estancia y por sala. Se compone de:

- una aplicación Android (Kotlin, Jetpack Compose) con cuatro perfiles: Fundación, Dirección, Educadora y Familia;
- un backend propio (Ktor + PostgreSQL), contenerizado con Docker y publicado por HTTPS a través de un túnel de Cloudflare;
- un módulo `shared/` con los modelos y las reglas de privacidad que usan tanto la app como el servidor.

El backlog tiene 21 requerimientos funcionales (RF-01 a RF-21): 14 validados por el socio formador y 7 propuestos por el equipo. Se organizan en 8 épicas y 36 historias de usuario. De los 21, en la versión 3.0.0 se cumplen 20 y uno queda fuera del alcance entregado: RF-11, la operación sin conexión (ver la sección 8.2).

Los flujos principales son:

- la educadora registra la llegada y la salida de cada niño, validando a la persona autorizada, y la bitácora del día;
- la familia ve en su teléfono el día y la semana de cada uno de sus hijos, y solo de ellos;
- las fotos solo las ven las familias de los niños que aparecen y con el permiso de imagen correspondiente, y cualquiera de esas familias puede borrarlas;
- la Dirección inscribe niños, crea cuentas y consulta el registro de actividad;
- la Fundación administra estancias y ve totales, nunca datos de un niño en particular.

1.3. Referencia al acta de acuerdos

El equipo no firmó con el socio formador un acta de acuerdos como documento aparte. Los acuerdos de alcance quedaron en el *Product Backlog* (versión validada en la sesión con el socio formador, Entrega 1). Ahí se separan los 14 requerimientos validados por la Fundación de los 7 propuestos por el equipo, que se marcan como *Nueva* hasta que el socio formador los valide. Ese backlog es la referencia de alcance de esta entrega. Todos los identificadores RF y HU de este documento son los del backlog.

2. Arquitectura final del sistema

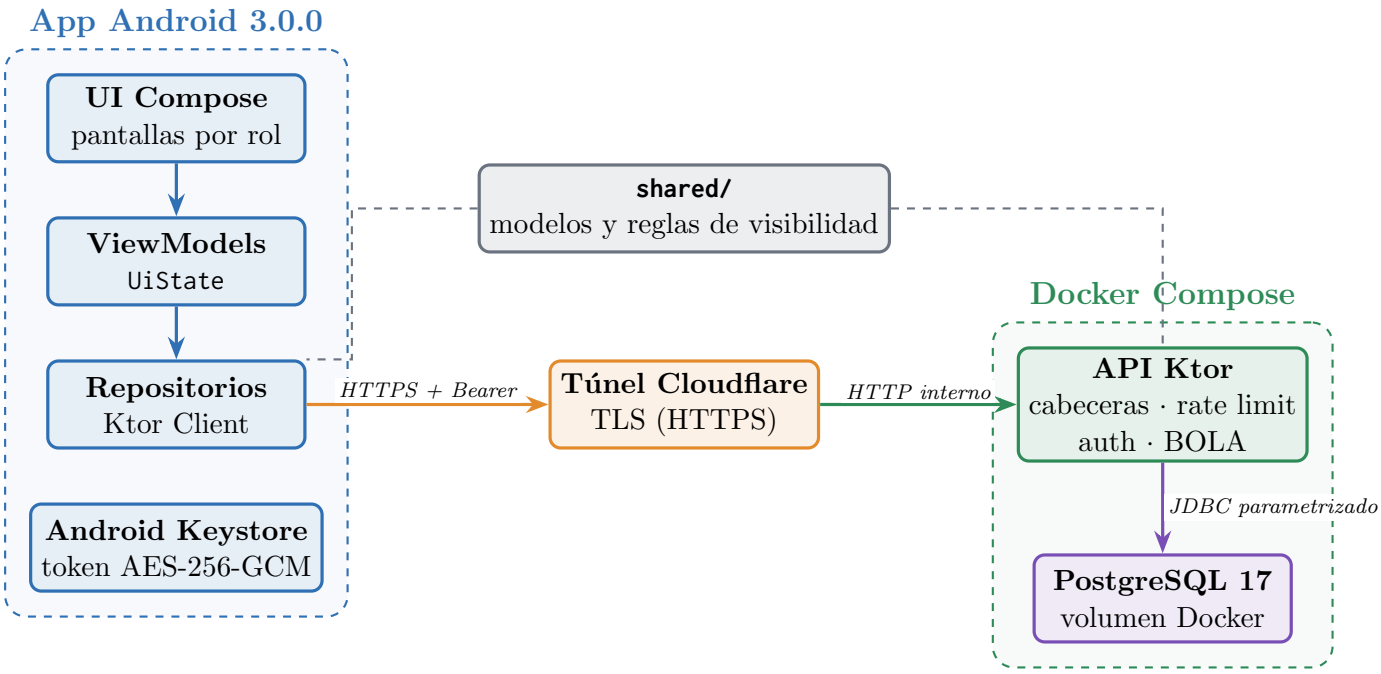


Figura 1: Capas y componentes de Sonrisas de Cristal. Las líneas punteadas indican el código compartido.

2.1. Capas y componentes

App Android. Arquitectura MVVM con una sola Activity. La capa `ui/` contiene las pantallas por rol (Educadora, Familia, Dirección y Fundación) y la navegación. Cada pantalla recibe un `UiState` (cargando, con datos, vacío o error) de su ViewModel. Los ViewModels llaman a repositorios que hablan con el API (`data/remoto`, Ktor Client). El token de sesión se guarda cifrado con AES-256-GCM, con una llave del Android Keystore.

Backend. Servidor Ktor (Kotlin, JVM 21). Expone 64 rutas REST, en JSON y con Bearer token. Cada petición pasa por cabeceras de seguridad, *rate limiting* por IP,

autenticación y autorización por rol y por objeto, que se calcula en el servidor a partir del usuario. Las rutas están separadas por dominio: `RutasNinos` (expediente, asistencia, bitácora e incidentes), `RutasContenido` (fotos, avisos, objetos perdidos y material), `RutasMensajes` y `RutasEstructura` (estancias, salas, cuentas y bloqueos).

Persistencia. PostgreSQL 17 en un volumen de Docker. Todas las consultas usan `PreparedStatement`. Los archivos (fotos y material) se guardan en la base como `BYTEA`, así que un respaldo de la base incluye todo.

`shared/`. Módulo Kotlin puro que comparten la app y el servidor: modelos, reglas de visibilidad (quién ve a qué niño), reportes y datos de demostración. Así la regla de privacidad es una sola y no se duplica.

Infraestructura. Docker Compose con tres servicios: `db`, `backend` (non-root, sistema de archivos de solo lectura, sin capabilities y con `no-new-privileges`) y `tunel` (`cloudflared`). El puerto del backend solo escucha en `127.0.0.1`; desde fuera solo se entra por el túnel HTTPS.

2.2. Decisiones técnicas principales

Decisión	Motivo y efecto
Backend propio en Ktor con `shared/`	La app y el servidor comparten modelos y reglas de visibilidad. Una regla de privacidad se escribe una vez y se prueba una vez.
Tokens opacos en lugar de JWT	El servidor guarda solo el SHA-256 del token. Cerrar sesión o retirar un acceso invalida el token de inmediato, cosa que un JWT sin lista de revocación no permite.
Autorización por objeto en el servidor	El servidor nunca confía en ids que mande el cliente: calcula los niños visibles a partir del usuario. A recursos ajenos responde 403 o 404 (BOLA/IDOR).
Hora del servidor como autoridad	El servidor fija la fecha de todo registro y limita la hora a su reloj. Cambiar el reloj del teléfono no permite fechar registros en otro día ni en el futuro.
Re-codificación de imágenes	Toda foto se valida por *magic bytes*, se decodifica y se vuelve a codificar. Así se eliminan EXIF, GPS y cualquier contenido agregado al archivo.

Decisión	Motivo y efecto
Sin registro público	Las cuentas solo las crean la Fundación (Dirección) y la Dirección (educadoras y familias). Nadie puede hacerse pasar por tutor de un niño ajeno.
Sin modo sin conexión	RF-11 se dejó fuera: sincronizar datos de menores sin conflictos ni pérdida de datos requería más tiempo que el disponible. Ver la sección 8.2.
Datos de demostración solo en debug	En release la app solo habla con el servidor; los datos de demostración viven en la base mediante una semilla idempotente.
Túnel de Cloudflare	Da HTTPS público sin abrir puertos ni administrar certificados. Usa HTTP/2 sobre TCP 443 porque la red del campus bloquea QUIC.

3. Matriz de trazabilidad y evidencia de pruebas

3.1. Niveles de prueba

Unitarias. Las reglas puras (visibilidad, diseño de la bitácora imprimible y contraseñas temporales) se prueban sin servidor ni Android: `DisenoBitacoraTest` en la app y los casos de regla de `RF02`/`RF07`.

Integración y API. Cada prueba de `backend/src/test` levanta el servidor Ktor completo (`testApplication`) con una base de datos real y lo ejercita por HTTP, como lo haría la app. La suite corre contra H2 en modo PostgreSQL y también contra un PostgreSQL 17 real en Docker (ver `docs/pruebas/resultados.txt`).

Seguridad. `SeguridadTest`, `SEGBloqueoCuentaTest`, `SEGArchivosTest`, `SEGRelojServidorTest` y `ARCOEliminacionTest` prueban que el API rechace peticiones maliciosas o no permitidas.

DAST. Escaneo con OWASP ZAP sobre las rutas del API (sección 4.4).

Sistema. Recorrido manual de punta a punta con el APK de release firmado en un dispositivo físico conectado al backend publicado por el túnel.

3.2. Resultados

Suite	Pruebas	H2 (PostgreSQL mode)	PostgreSQL 17
RF01–RF15 (por requerimiento)	40	40/40	40/40
SeguridadTest (S01–S27)	27	27/27	en H2*
SEGBloqueoCuentaTest	4	4/4	en H2*
SEGArchivosTest	6	6/6	6/6
SEGRelojServidorTest	7	7/7	7/7
ARCOEliminacionTest	3	3/3	3/3
ServidorTest y DbTest	10	10/10	10/10
Backend total	**97**	**97/97**	**64/64**
App: DisenoBitacoraTest y ArmadoBitacoraTest	7	7/7 (JVM)	

* Estas dos suites crean su propia base con un reloj o un proxy configurados para la prueba, así que siempre corren en H2. Las otras 64 pruebas con base de datos corrieron también contra PostgreSQL 17 real.

3.3. Matriz de trazabilidad

Los identificadores son los del backlog. Las pruebas automatizadas se nombran `Clase.prueba`. Las pruebas manuales (PM) son recorridos en el teléfono con el APK de release; se describen en la sección 5.

ID	Descripción	Criterio de aceptación	Prueba o evidencia	Resultado	Estado
RF-01	Acceso con perfil propio	Cada rol entra a su navegación; contraseña incorrecta no entra	`RF01AccesoPerfilesTest` (4)	Aprobada	Cumplido
RF-02	Lista de niños de la sala	La educadora ve solo su sala	`RF02ListaNinosSalaTest` (4)	Aprobada	Cumplido
RF-03	Entrada y salida	Hora, quién entrega y quién recoge; sin duplicados	`RF03AsistenciaTest` (4), `SEGRelojServidorTest.laAsistencia...`	Aprobada	Cumplido

ID	Descripción	Criterio de aceptación	Prueba o evidencia	Resultado	Estado
RF-04	Bitácora del día	Comida, siesta y notas firmadas por la educadora	`RF04BitacoraTest` (4)	Aprobada	Cumplido
RF-05	Registro para varios niños	Un mismo evento se guarda para todos los niños elegidos	PM-05: registro grupal en el teléfono y verificación en la cuenta de cada familia	Correcto	Cumplido
RF-06	Vista del familiar	La familia ve el día que registró la educadora	`RF06FamiliaVeElDiaTest` (4)	Aprobada	Cumplido
RF-07	Cada familia ve solo a sus hijos	Un niño ajeno no aparece ni por id	`RF07FamiliaSoloSus HijosTest` (4), `SeguridadTest.S11`	Aprobada	Cumplido
RF-08	Avisos del centro	Aviso publicado con hora del servidor, visible para la estancia	`SEGRelojServidorTest.elAviso. . .`, PM-08	Aprobada	Cumplido
RF-09	Fotografía de la actividad	Solo la ven las familias de los niños etiquetados	`RF09FotosVisiblesTest` (4), `ServidorTest.laImagenDeUnaFoto. . .`	Aprobada	Cumplido
RF-10	Borrar una foto	La familia la borra para todos, con su imagen	`RF10EliminarFotoTest` (4)	Aprobada	Cumplido
RF-11	Funciona sin internet	Registrar sin conexión y sincronizar sin duplicados	Sin prueba: no se implementó	No aplica	Pendiente
RF-12	Resumen semanal	Semana del niño e impresión con el formato oficial	`DisenoBitacoraTest` (7), PM-12	Aprobada	Cumplido
RF-13	Desbloqueo con huella	La app se bloquea en segundo plano y abre con la huella	PM-13 en dispositivo con lector	Correcto	Cumplido

ID	Descripción	Criterio de aceptación	Prueba o evidencia	Resultado	Estado
RF-14	Quién hizo qué	Capturas, consultas y exportaciones registradas; solo las lee la Dirección	`RF14RegistroActividadTest` (4)	Aprobada	Cumplido
RF-15	Inscripción y expediente	Solo personas autorizadas del expediente entregan o recogen	`RF15SalidaPersonaAutorizadaTest` (4)	Aprobada	Cumplido
RF-16	Autorización de imagen	No se etiqueta a un niño sin el permiso del uso elegido	Validación en `POST / fotos`; PM-16	Correcto	Cumplido
RF-17	Varias estancias	Una Dirección no toca otra estancia; la Fundación no ve niños	`SeguridadTest.S12`, `ServidorTest.la FundacionNoRecibe...`	Aprobada	Cumplido
RF-18	Baja y eliminación	Baja con 30 días de gracia y borrado completo	`SeguridadTest.S20/S21`, `ARCOEliminacionTest` (3)	Aprobada	Cumplido
RF-19	Incidentes	Incidente con folio y hora del servidor, visible a la familia	`SEGRelojServidorTest.elIncidente...`, PM-19	Aprobada	Cumplido
RF-20	Reporte de asistencia	Por sala y mes; la Fundación solo recibe totales	`ServidorTest.la FundacionNoRecibe...`, PM-20	Aprobada	Cumplido
RF-21	Horario de actividades	La familia ve el horario de la sala de su hijo	PM-21	Correcto	Cumplido
RNF-SEG-01	Sin secretos en texto plano	La base no guarda contraseñas ni tokens legibles	`SeguridadTest.S06`	Aprobada	Cumplido
RNF-SEG-02	Sesiones robustas	Token ausente, inventado, alterado o vencido responde 401	`SeguridadTest.S01--S05`	Aprobada	Cumplido

ID	Descripción	Criterio de aceptación	Prueba o evidencia	Resultado	Estado
RNF-SEG-03	Sin inyección SQL	Las inyecciones no autentican, no borran tablas ni filtran datos	`SeguridadTest.S08--S10`	Aprobada	Cumplido
RNF-SEG-04	Control de privilegios	Roles sin permiso no borran ni crean cuentas	`SeguridadTest.S13/S14/S26`	Aprobada	Cumplido
RNF-SEG-05	Rate limiting y bloqueo	429 por exceso de peticiones; 3 IPs distintas bloquean 3 días; solo la Fundación desbloquea	`SeguridadTest.S15--S19`, `SEGBloqueoCuentaTest` (4)	Aprobada	Cumplido
RNF-SEG-06	Archivos seguros	Solo JPEG/PNG reales de 10 MB o menos; sin EXIF ni GPS	`SEGArchivosTest` (6+)	Aprobada	Cumplido
RNF-SEG-07	Hora no falsificable	Fechas de 2001 o 2099 enviadas por el cliente se ignoran	`SEGRelojServidorTest` (7)	Aprobada	Cumplido
RNF-SEG-08	Cabeceras de seguridad	HSTS, CSP, nosniff y X-Frame-Options en cada respuesta	`SeguridadTest.S25`, ZAP	Aprobada	Cumplido
RNF-PRIV-01	Derecho de cancelación	Borrar un expediente elimina sus datos, imágenes y la cuenta de la familia	`ARCOEliminacionTest` (3)	Aprobada	Cumplido

3.4. Evidencia

Todos los archivos de evidencia, con su ruta desde la raíz del repositorio y su enlace en GitHub, están en el apéndice A.

- Reporte JUnit del backend: `docs/pruebas/reporte-backend/index.html` y resumen por clase en `docs/pruebas/resultados.txt`.
- Verificación del despliegue por el túnel (cabeceras, login de cada rol y un dato

capturado por la educadora y leído por la familia): `docs/seguridad/verificacion-despliegue.txt`.

- Reporte de OWASP ZAP: `docs/seguridad/zap/`.
- Capturas por rol: `evidencia-pruebas/`.

4. Evidencia de los controles de seguridad

4.1. Controles en la API

Control	Implementación	Evidencia
Sesiones e identidad	Contraseñas con PBKDF2-HMAC-SHA256 (210 000 iteraciones, sal de 16 bytes, comparación en tiempo constante). Tokens opacos de 256 bits; el servidor solo guarda su SHA-256. Vencen a los 30 días y se revocan al cerrar sesión o al retirar el acceso. En el teléfono, el token se guarda cifrado con AES-256-GCM, con una llave del Android Keystore.	`Seguridad.kt`; `SeguridadTest.S01--S07`
Autorización por rol y objeto (BOLA/IDOR)	Cada ruta calcula del lado del servidor qué estancia, sala y niños puede ver el usuario (`Visibilidad` en `shared/`). Un id ajeno responde 403 o 404 y nunca devuelve datos.	`S11--S14`, `RF02`, `RF07`, `RF09`
Operaciones destructivas	Para borrar un expediente se exige que antes esté dado de baja y que lo pida la Dirección de esa estancia. Borrar fotos y retirar accesos se valida contra el dueño del recurso. Toda escritura queda en el registro de actividad.	`S13`, `S20`, `S21`, `RF10`, `RF14`
Entrada parametrizada y validada	Todas las consultas usan `PreparedStatement` con `?`. Fechas, meses, longitudes y tipos se validan y responden 400, no 500.	`S08--S10`, `S22`, `S23`

Control	Implementación	Evidencia
Rate limiting y bloqueo	Límite general de 1000 peticiones por minuto por IP; 10 logins por minuto, 20 altas de cuenta y 30 subidas por minuto. Con 5 fallos en 15 minutos la cuenta se bloquea 15 minutos. Con fallos desde 3 IPs distintas en 24 horas se bloquea 3 días: el bloqueo se guarda en la base, se registra en la bitácora y solo la Fundación lo quita (`GET /bloqueos`, `POST /cuentas/{id}/desbloqueo` y pantalla de inicio de la Fundación). La IP real se toma del proxy confiable, así que `X-Forwarded-For` no se puede falsificar.	`S15--S19`, `SEGBloqueoCuentaTest`
Archivos	El tipo se detecta por *magic bytes* (solo JPEG y PNG para fotos), la imagen se decodifica completa, con un tope de 40 megapíxeles, y se vuelve a codificar sin metadatos (EXIF, GPS, XMP ni comentarios). Límite de 10 MB. La app también vuelve a comprimir antes de subir.	`SEGArchivosTest`
Hora del servidor	El servidor fija la fecha de cada registro y limita la hora a su propio reloj. Una fecha falsificada en el teléfono no llega a la base. No existe creación sin conexión que pueda traer horas del dispositivo.	`SEGRelojServidorTest`
Tráfico cifrado y cabeceras	La app en release solo acepta HTTPS (sin `cleartext`). El túnel de Cloudflare termina TLS. Cada respuesta lleva `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `Content-Security-Policy: default-src 'none'`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer` y `Cache-Control: no-store`.	`S25`; `verificacion-despliegue.txt`

4.2. Infraestructura y despliegue seguro

- El backend se construye en dos etapas (JDK para compilar, JRE para correr) y corre como el usuario `sonrisas`, no como root.
- En Compose el contenedor es `read_only`, con `tmpfs` solo en `/tmp`, `cap_drop: ALL` y `no-new-privileges`.
- Los secretos (`POSTGRES_PASSWORD` y `CLAVE_FUNDACION`) llegan por variables de entorno desde un `.env` que está en `.gitignore`. Sin `POSTGRES_PASSWORD`, Compose no arranca.
- El puerto 8080 solo escucha en `127.0.0.1`. El acceso por internet es por `cloudflared` con HTTP/2 sobre TCP 443. La base no expone puertos.

4.3. Auditoría OWASP

La auditoría tomó como referencia la edición vigente del OWASP Top 10 (2025), verificada en https://owasp.org/Top10/2025/, y el OWASP API Security Top 10 (2023). Para cada categoría se revisó el código y se buscó una prueba que lo demuestre.

Categoría	**Resultado en Sonrisas de Cristal**
A01 Broken Access Control / API1 BOLA / API5 BFLA	Autorización por objeto y por función en el servidor; pruebas S11–S14 y RF02/RF07/RF09. En la auditoría se encontró y corrigió un caso: una imagen huérfana se podía descargar solo con su id. Ahora el acceso siempre se valida por sala (`ARCOEliminacionTest`).
A02 Security Misconfiguration / API8	Cabeceras de seguridad, contenedor endurecido, puerto local y errores sin trazas internas (`StatusPages`). ZAP no reportó configuraciones inseguras de riesgo medio o alto (sección 4.4).
A03 Software Supply Chain Failures	Versiones fijas en Gradle (catálogo de versiones) y en Docker (`eclipse-temurin:21`, `postgres:17` y `cloudflared:2026.9.3`, que antes era `latest` y se fijó en esta auditoría). No se agregaron dependencias para los controles nuevos: la limpieza de imágenes usa `javax.imageio`.
A04 Cryptographic Failures	PBKDF2 con 210 000 iteraciones, tokens de 256 bits con `SecureRandom`, TLS en tránsito y AES-GCM en el teléfono.
A05 Injection	Consultas parametrizadas en todas las rutas; S08–S10.
A06 Insecure Design / API6	Sin registro público, borrado en dos pasos (baja y después eliminación) y permiso de imagen validado en el servidor.

Categoría	Resultado en Sonrisas de Cristal
A07 Authentication Failures / API2	Rate limiting, bloqueo por cuenta y multi-IP, mensaje de error que no delata si la cuenta existe (S07 y S18) y contraseñas temporales de alta entropía (S27).
A08 Software or Data Integrity Failures	APK firmado; la hora y la autoría de cada registro las fija el servidor (`SEGRelojServidorTest`, `RF04`).
A09 Security Logging & Alerting Failures	Registro de actividad (RF-14) más registro del servidor de bloqueos y desbloqueos. No hay alertas automáticas: queda como recomendación de implantación.
A10 Mishandling of Exceptional Conditions / API4	Las entradas inválidas responden 400 y no 500 (S22). Límites de tamaño (413) y de tasa (429). El servidor falla cerrado: sin sesión válida responde 401.

4.4. Escaneo dinámico con OWASP ZAP

Se corrió `zap-api-scan.py` (imagen `ghcr.io/zaproxy/zaproxy:stable`) contra el backend desplegado con Docker Compose, en la misma red de contenedores. Se usó una especificación OpenAPI con todas las rutas, generada desde el código (`docs/seguridad/zap/openapi.json`), y un token Bearer de la Dirección para que el escaneo alcanzara las rutas autenticadas. El reporte completo está en `docs/seguridad/zap/reporte-zap.html`.

Riesgo	**Alerta**	**Instancias**	**Justificación o mitigación**
Alto / Medio	Ninguna	0	No aplica
Bajo	Cross-Origin-Resource-Policy Header Missing	1 → 0	Corregido: `Cross-Origin-Resource-Policy: same-origin` en `CabecerasSeguras`; se volvió a escanear
Informativo	A Client Error response code was returned	4385	Esperado: son los 400, 403, 404 y 429 con que el API rechaza entradas inválidas, accesos no permitidos y exceso de peticiones; ningún 500
Informativo	Non-Storable Content	5	Intencional: `Cache-Control: no-store` para que ni proxies ni teléfonos guarden datos de niños en caché

Cuadro 5: Alertas de OWASP ZAP (escaneo autenticado). Resumen: FAIL-NEW 0, WARN-NEW 0, PASS 120.

Durante el escaneo autenticado el backend respondió 3867 veces con 429 (límite general de 1000 peticiones por minuto por IP), 150 con 403 (la Dirección intentó acciones de otro rol), 595 con 400, ninguna con 401 (la sesión siguió válida) y ninguna con 500. El rate limiting también recortó parte del escaneo activo. Por eso, la inyección SQL, el IDOR y la escalada de privilegios se verifican además con las pruebas deterministas S08–S14 y `SEGBloqueoCuentaTest`, que no dependen del ritmo del escáner. Antes de este escaneo se corrió uno sin sesión (`antes-reporte-zap.*`), donde apareció la única alerta baja. El detalle está en `docs/seguridad/zap/hallazgos.md`.

La verificación del despliegue por el túnel HTTPS está en `docs/seguridad/verificacion-despliegue.txt`: cabeceras presentes, contenedor con el usuario `sonrisas` (uid 999), `ReadonlyRootfs`, `CapDrop=ALL`, `no-new-privileges` y puerto 8080 solo en `127.0.0.1`. El túnel rápido de Cloudflare también contesta por `http://` sin redirigir. La app release no lo usa porque bloquea el tráfico en claro, y HSTS obliga a HTTPS en clientes que lo respetan.

5. Manual de usuario

5.1. Entrar a la app

1. Abre Sonrisas de Cristal y escribe el usuario y la contraseña que te dio la Dirección o la Fundación.

2. Si es la primera vez en ese teléfono, la Dirección configura el servidor: mantiene presionado el logo y escribe la dirección `https://` del servidor.

3. Si escribes mal la contraseña varias veces, la cuenta se bloquea un tiempo. Si los intentos vienen de varios lugares, se bloquea 3 días o hasta que la Fundación la desbloquee.

4. En Perfil puedes activar el desbloqueo con huella (PM-13). La app se bloquea al pasar a segundo plano o tras 5 minutos sin uso.

5. El aviso de privacidad está en Perfil → *Acerca de y aviso de privacidad*.

Cuentas de demostración (contraseña `demo`): `educadora`, `familia`, `direccion` y `fundacion`. Hay más cuentas por sala y por estancia, listadas en el `README`.

5.2. Educadora

Llegada y salida (RF-03, RF-15). En *Sala*, toca al niño y luego *Llegada* o *Salida*. Elige de la lista a la persona que lo entrega o lo recoge: solo aparecen las personas autorizadas del expediente. La hora la propone la app y el servidor la valida.

Bitácora (RF-04). En la ficha del niño, usa las acciones rápidas: comida, siesta, baño, ánimo y nota.

Registro grupal (RF-05, PM-05). Desde *Sala*, elige *Registro grupal*, marca a los niños y guarda un solo evento para todos.

Incidente (RF-19, PM-19). En la ficha, *Incidente*: tipo, parte del cuerpo y acción tomada. Se genera un folio y la familia lo ve en *Hoy*.

Fotos (RF-09, RF-16, PM-16). En *Fotos*, toma la foto con la cámara, escribe una descripción y etiqueta a los niños. La app no deja etiquetar a un niño sin el permiso de imagen para ese uso.

Mensajes y avisos (RF-08, PM-08). Chat de la sala, mensajes directos con las familias y avisos de la estancia.

5.3. Familia

Hoy y Semana (RF-06, RF-12, PM-12). Muestran la llegada, la salida, quién lo recogió, las comidas, las siestas, los incidentes y el resumen semanal de cada uno de tus hijos. La bitácora se puede imprimir o compartir en PDF.

Fotos (RF-09, RF-10). Ves solo las fotos donde aparece tu hijo. Con *Eliminar*, la foto se borra para todas las familias.

Horario (RF-21, PM-21), material y objetos perdidos. Desde la tarjeta de la sala en *Hoy*.

5.4. Dirección

Inscripción (RF-15). En *Niños*, *Inscribir*: datos del niño, salud, tutores, personas autorizadas y permisos de imagen. Al crear la cuenta de un tutor, el usuario y la contraseña temporal se muestran una sola vez.

Baja y eliminación (RF-18). En el expediente, *Dar de baja*. El expediente se borra solo a los 30 días, o al momento con *Eliminar definitivamente*. Así se atiende una solicitud de cancelación del aviso de privacidad.

Reportes y registro (RF-14, RF-20, PM-20). En *Más*: reporte de asistencia por sala y mes, y el registro de quién hizo qué.

Personal y salas. Alta de educadoras, asignación de sala y retiro de acceso.

5.5. Fundación

- *Inicio* muestra el resumen de todas las estancias y su asistencia, sin nombres de niños (RF-17, RF-20).
- Desde ahí se registran y editan estancias y se crea la cuenta de Dirección de cada una.
- Si hay cuentas bloqueadas, aparece la sección *Cuentas bloqueadas*, con un botón *Desbloquear* por cuenta.

6. Manual de instalación y ejecución

Esta sección basta para que una persona externa levante el sistema completo, instale la app y corra las pruebas. Todos los comandos se ejecutan desde la raíz del repositorio.

6.1. Requisitos

- **Backend:** Docker 24 o superior con Docker Compose. No hace falta instalar Java ni PostgreSQL.
- **App:** un teléfono o emulador con Android 8.0 o superior.
- **Solo para compilar o correr pruebas:** el JDK que trae Android Studio.

6.2. Levantar el backend (un comando)

```
$ git clone https://github.com/santihdzs/sonrisas-de-cristal.git
$ cd sonrisas-de-cristal
$ ./levantar.sh
```

`levantar.sh` hace todo lo siguiente:

- crea el `.env` a partir de `.env.example`, que ya trae valores listos para la demostración;
- construye y levanta PostgreSQL, el backend y el túnel HTTPS de Cloudflare;
- al arrancar, carga los datos de demostración en la base;
- imprime la dirección pública `https://. . . .trycloudflare.com` que usará la app.

La salida termina así:

```
Backend local:                http://127.0.0.1:8080
Backend público (para la app): https://<palabras>.trycloudflare.com
Cuentas de demostración: educadora, familia, direccion, fundacion (contraseña: demo)
```

En Windows (PowerShell), el equivalente es:

```
$ Copy-Item .env.example .env
$ docker compose --profile tunel up -d --build
$ docker compose logs tunel
```

El último comando muestra la dirección `https://` del túnel.

6.3. Instalar y conectar la app

1. Descarga el APK firmado (sección 10) y ábrelo en el teléfono. Acepta *instalar apps de origen desconocido* si te lo pide.

2. En la pantalla de inicio de sesión, mantén presionado el logo y pega la dirección `https://` que imprimió `levantar.sh`. La dirección del túnel rápido cambia cada vez que el túnel se reinicia; en ese caso se repite este paso.

3. Entra con una cuenta de demostración. Todas usan la contraseña `demo`:

Rol	**Usuarios**
Fundación (administración)	`fundacion`
Dirección	`direccion`, `direccion.vv`
Educadora	`educadora`, `educadora.lactantes`, `educadora.preescolar`, `educadora.vv.*`
Familia	`familia`, `familia.papa`, `familia.paola`

6.4. Operación

Recargar la demostración con la fecha de hoy (borra los datos y vuelve a sembrar; se niega a correr con `DATOS_DEMO=false`):

```
$ docker compose run --rm --no-deps backend /app/bin/backend reiniciar-demo
```

Detener todo, sin borrar datos:

```
$ docker compose --profile tunel down
```

Variables de `.env` (todas tienen valor en `.env.example`):

- `POSTGRES_PASSWORD`: contraseña de la base. En producción se cambia por una aleatoria (`openssl rand -hex 24`).
- `DATOS_DEMO`: `true` carga la demostración. En producción, `false`.
- `CLAVE_FUNDACION`: contraseña inicial de la cuenta `fundacion` cuando `DATOS_DEMO=false`; mínimo 12 caracteres.
- `PROXY_CONFIABLE`: `true` detrás del túnel.
- `HOST_IP`: `127.0.0.1`, para que el puerto no quede abierto a la red.

6.5. Correr las pruebas

```
$ export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"
$ ./gradlew -p backend test          # 97 pruebas: API, integracion y seguridad
$ ./gradlew :app:testDebugUnitTest   # unitarias de la app
```

Para correr la misma suite contra PostgreSQL real, se levanta uno desechable y se pasa su URL:

```
$ docker run -d --rm --name pg-pruebas -p 127.0.0.1:55432:5432 \
    -e POSTGRES_USER=sonrisas -e POSTGRES_PASSWORD=pruebas -e POSTGRES_DB=sonrisas
    postgres:17
$ PRUEBAS_DATABASE_URL=postgres://sonrisas:pruebas@127.0.0.1:55432/sonrisas \
    ./gradlew -p backend test
$ docker stop pg-pruebas
```

El reporte HTML queda en `backend/build/reports/tests/test/index.html`.

6.6. Compilar el APK de release firmado

La llave de firma no está en el repositorio. Se crea una vez y se describe en `keystore.properties`, que está en `.gitignore`:

```
$ keytool -genkeypair -keystore ~/sonrisas-release.jks -alias sonrisas \
    -keyalg RSA -keysize 3072 -validity 10000
$ printf 'storeFile=%s\nstorePassword=...\nkeyAlias=sonrisas\nkeyPassword=...\n' \
    ~/sonrisas-release.jks > keystore.properties
$ ./gradlew :app:assembleRelease
```

El APK queda en `app/build/outputs/apk/release/app-release.apk`.

7. Plan de implantación

Versión entregada. App 3.0.0 (`versionCode` 3) y backend del tag `v3.0-entrega-final` del repositorio.

Canal de distribución. APK de release firmado, publicado en el repositorio (`entrega/`), que la Dirección instala en los teléfonos o tabletas de cada sala. Las familias reciben el mismo APK por enlace. La publicación en Google Play (prueba cerrada) queda como siguiente paso; requiere la cuenta de desarrollador de la Fundación.

Configuración necesaria. Un equipo o VPS de la Fundación con Docker y un archivo `.env` con `POSTGRES_PASSWORD` aleatoria, `DATOS_DEMO=false` y `CLAVE_FUNDACION` de 12

caracteres o más. En operación real se recomienda un túnel con nombre de Cloudflare (dominio fijo) en lugar del túnel rápido, cuya dirección cambia al reiniciar. En la app, la dirección del servidor se fija al compilar (`-Pservidor=https://...`) o se configura una vez manteniendo presionado el logo.

Procedimiento de despliegue.

1. Clonar el tag `v3.0-entrega-final`, copiar `.env.example` a `.env` y ajustar los valores de producción: `POSTGRES_PASSWORD` aleatoria, `DATOS_DEMO=false` y `CLAVE_FUNDACION`.
2. Correr `./levantar.sh` (sección 6).
3. Confirmar que `GET /` responde por `https` con las cabeceras de seguridad.
4. Entrar como `fundacion`, registrar las estancias y crear la cuenta de Dirección de cada una.
5. Cada Dirección registra salas y educadoras e inscribe a los niños.
6. Instalar el APK y configurar el servidor en cada dispositivo.

Carga y migración de datos. El socio formador **no tiene datos previos que migrar**: hoy la información vive en bitácoras en papel y en chats de WhatsApp, y no conviene trasladar fotos de menores desde teléfonos personales. La carga inicial se hace desde la app. La semilla de producción crea solo la cuenta `fundacion`, y cada estancia captura a su personal y sus inscripciones conforme los niños se reinscriben, con el consentimiento por escrito del aviso de privacidad. Los datos de demostración (`DATOS_DEMO=true`) solo se usan para capacitación y para la presentación.

Respaldo. Diario, con el contenedor de la base. Como fotos y material se guardan en la base, el respaldo incluye los archivos.

```
$ docker compose exec db pg_dump -U sonrisas sonrisas > respaldo.sql
```

Rollback. Si la nueva versión falla, se regresa a la anterior con el respaldo previo y se reinstala el APK anterior (desinstalando antes, porque Android no permite bajar el `versionCode`):

```
$ docker compose stop backend
$ docker compose exec -T db psql -U sonrisas sonrisas < respaldo.sql
$ git checkout <tag-anterior>
$ docker compose up -d --build
```

Como el esquema solo usa `CREATE TABLE IF NOT EXISTS`, una versión anterior arranca sobre la misma base sin migraciones inversas; la tabla nueva `bloqueos` simplemente no se usa.

8. Administración final del proyecto

8.1. Backlog final

El backlog final tiene 21 requerimientos funcionales (RF-01 a RF-21), 36 historias de usuario en 8 épicas (E1 Acceso y perfiles a E8 Confiabilidad y trazabilidad), tareas técnicas y requerimientos no funcionales. Los RNF de seguridad agregados en esta entrega son RNF-SEG-01 a 08 y RNF-PRIV-01 (sección 8.2). El tablero se administra en Microsoft Planner:

- Plan de Planner: enlace al tablero (Teams, NeuralHub).
- Tareas técnicas principales: backend Ktor y PostgreSQL; integración de la app con el API; endurecimiento P1–P8 (cabeceras, rate limiting, bloqueo, Docker, túnel, token cifrado); rediseño de UI; pruebas por RF; limpieza de imágenes; hora del servidor; bloqueo multi-IP; aviso de privacidad y borrado ARCO; escaneo ZAP.
- RNF: no almacenar secretos en texto plano; los cuatro estados de UI (cargando, datos, vacío y error); no mostrar datos de otro niño, familia o estancia; pantalla principal en menos de 2 segundos.

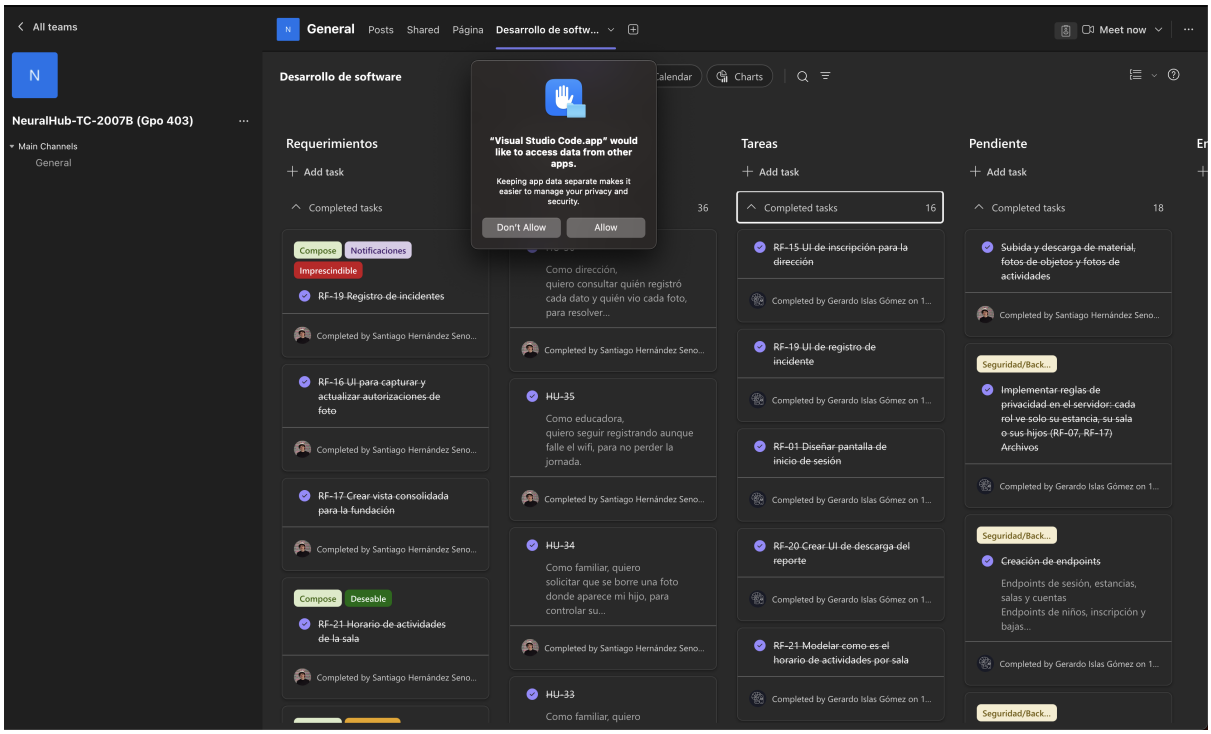


Figura 2: Tablero final de Planner.

8.2. Estado de los requerimientos

- **Terminados (20):** RF-01 a RF-10 y RF-12 a RF-21, con la evidencia de la matriz de trazabilidad.
- **Fuera del alcance entregado (1):** RF-11, operación sin conexión. El backlog lo dejó abierto con preguntas sin resolver con el socio formador (qué acciones no deben permitirse sin conexión y qué dato gana en un conflicto), y no se comprometió su implementación. La app es solo en línea, y por eso el servidor puede ser la única fuente de verdad, incluida la hora.
- **Historias parciales:** HU-05, recuperar la contraseña sin la Dirección, no se implementó: las familias no usan correo de forma confiable. Hoy la Dirección genera una contraseña temporal nueva. El cambio obligatorio de contraseña en el primer acceso también queda pendiente.

8.3. Bitácora de hitos

Fecha	Hito	Cumplimiento
22 sep 2026	Inicio del repositorio, arquitectura MVVM y módulo `shared/`	En fecha
Entrega 1	Backlog con 21 RF, 8 épicas y 36 HU, validado con el socio formador; Presentación 1	En fecha
2 oct 2026	Entrega 2: prototipo funcional con datos en memoria	En fecha
3–14 oct 2026	Backend Ktor + PostgreSQL e integración de la app con el API	En fecha
19–20 oct 2026	Endurecimiento de seguridad P1–P8 y rediseño de UI (PR #2 y #3)	En fecha
21 oct 2026	Pruebas por RF y pulido para la demostración	En fecha
22 oct 2026	Entrega final: bloqueo multi-IP, archivos, hora del servidor, aviso de privacidad, borrado ARCO, ZAP y APK firmado	En fecha, con RF-11 fuera

8.4. Desviaciones

- **RF-11 (sin conexión) no se implementó.** Causa: el backend propio y la seguridad del API consumieron el tiempo previsto, y las reglas de conflicto nunca se definieron con el socio formador.
- **Los datos de demostración pasaron de la app a la base.** En la Entrega 2 la app usaba datos en memoria. En la final, la versión release solo consume el API, y la demostración vive en PostgreSQL mediante una semilla idempotente.
- **El trabajo se concentró al final.** La seguridad y la documentación se cerraron en los últimos días. Ver la retrospectiva.

8.5. Riesgos

Riesgo	Acción tomada	Resultado
Fuga de fotos o datos de menores	Autorización por objeto en el servidor, permisos de imagen y limpieza de metadatos	Mitigado; pruebas RF07, RF09, SEG y ARCO
Red del campus bloquea el túnel (QUIC)	`cloudflared` con HTTP/2 sobre TCP 443	Resuelto
Fuerza bruta contra cuentas	Rate limiting y bloqueo por cuenta y multi-IP	Mitigado; S15–S19
Servidor caído durante la presentación	Video de respaldo y servidor local con emulador	Plan de contingencia listo
Dirección del túnel rápido cambia	Configuración del servidor en la app (mantener presionado el logo)	Aceptado; en producción, túnel con nombre
Datos inconsistentes entre teléfonos con relojes distintos	Hora del servidor como autoridad	Resuelto; `SEGReloj` `ServidorTest`

8.6. Distribución del trabajo

La evidencia está en el historial de `git` (autores de los commits y pull requests) y en las asignaciones de Planner.

- **Santiago Hernández Senosiain:** arquitectura, app Android, backend Ktor, integración de la app con el API, controles de seguridad finales (bloqueo multi-IP, archivos, hora del servidor, borrado ARCO), despliegue, escaneo ZAP y documentación técnica.
- **Gerardo Islas Gómez:** endurecimiento de seguridad P1–P8 (cabeceras, contraseñas temporales, token cifrado en el teléfono, Docker endurecido, secretos en `.env`, túnel HTTPS) y pruebas de seguridad del API (`SeguridadTest`, PR #2, rama `seguridad`).
- **Gilberto Olavarrieta Sánchez:** diseño de la aplicación: sistema visual, componentes compartidos y las vistas de todas las pantallas por rol (frontend en Compose, PR #3, rama `rediseno-ui`).
- **Franco Salazar Guerrero:** redacción del aviso de privacidad y de las consideraciones éticas y legales (criterios verificables, clasificación de datos y dilemas éticos), y revisión de que el aviso corresponda con lo que hace la app.

8.7. Retrospectiva

Qué funcionó.

- Compartir el módulo `shared/` entre la app y el servidor. La regla «cada familia ve solo a sus hijos» se escribió una vez y se probó una vez.
- Nombrar las pruebas por RF. La matriz de trazabilidad salió casi sola de los nombres de las clases de prueba.
- Ponerle número a cada cambio de seguridad (P1–P8) y dejarlo en el commit, lo que hizo fácil explicar cada control.

Qué no funcionó.

- Construir el prototipo con datos en memoria en la app (Entrega 2) obligó a mantener dos implementaciones de cada repositorio, la falsa y la remota, hasta el final.
- Varios huecos se descubrieron hasta la auditoría final: imágenes que quedaban huérfanas al borrar un expediente y fechas que mandaba el cliente.
- El trabajo de seguridad y documentación quedó concentrado en los últimos días, y la distribución entre integrantes fue desigual.

Qué cambiaríamos.

- Levantar el backend mínimo desde el Sprint 1 y escribir el aviso de privacidad desde el inicio, para diseñar el borrado de datos junto con el modelo y no al final.
- Asignar en Planner a cada integrante un módulo con pruebas propias desde la primera semana, para que la contribución quede visible en `git` y no dependa de una sola persona.

- Cerrar con el socio formador las preguntas abiertas del backlog, por ejemplo las de RF-11, antes de comprometer el alcance.

9. Consideraciones éticas y legales

Ética, moral y ley en este proyecto.

- **La ley** fija mínimos exigibles. Aquí aplican la LFPDPPP (DOF 20-03-2025), la Ley General de los Derechos de Niñas, Niños y Adolescentes (LGDNNA) y el Código Penal Federal en materia de acceso ilícito a sistemas (arts. 211 bis 1 a 211 bis 7).
- **La moral** son las convicciones de la comunidad de la estancia. Por ejemplo, que las fotos de un niño no circulen fuera del centro, aunque ninguna norma lo diga con esas palabras.
- **La ética** es la reflexión del equipo para decidir cuando la ley no alcanza o cuando dos valores chocan, como la transparencia frente a la privacidad.

Cada criterio que salió de esas tres fuentes se convirtió en un requisito con prueba.

9.1. Criterios éticos y legales convertidos en requisitos verificables

ID	Requisito verificable	Criterio de aceptación	Prueba o evidencia	Ref. matriz
RF-07	(Ley: LGDNNA art. 76) Una familia solo puede obtener datos de sus propios hijos	Pedir por id el expediente, los tutores o el historial de un niño ajeno no devuelve datos	`RF07FamiliaSoloSusHijosTest`, `SeguridadTest.S11`	Matriz, fila RF-07
RF-16	(Moral y ética) No se puede etiquetar en una foto a un niño sin el permiso de imagen del uso elegido	`POST /fotos` con un niño sin permiso responde 403 y no guarda la foto	Validación en el servidor (`RutasContenido.kt`); PM-16	Matriz, fila RF-16
RF-10	(Ética) Cualquier familia que aparece en una foto puede borrarla para todos	La foto y su imagen dejan de existir para todas las familias	`RF10EliminarFotoTest`	Matriz, fila RF-10

ID	Requisito verificable	Criterio de aceptación	Prueba o evidencia	Ref. matriz
RNF-PRIV-01 / RF-18	(Ley: LFPDPPP arts. 24 y 25, cancelación) Eliminar un expediente borra sus datos de punta a punta	Tras eliminarlo, no quedan fotos solo de él, imágenes, asistencias, incidentes, registros ligados ni la cuenta y los chats de su familia	`ARCOEliminacionTest` (3)	Matriz, filas RF-18 y RNF-PRIV-01
RNF-SEG-06	(Ética: minimización, LFPDPPP art. 5) Ninguna foto conserva metadatos de ubicación	El archivo guardado no contiene EXIF, GPS ni el marcador APP1	`SEGArchivosTest.laFotoSeGuardaSinExifNiGps`	Matriz, fila RNF-SEG-06
RNF-SEG-01	(Ley: LFPDPPP art. 18, medidas de seguridad) Sin secretos en texto plano	La base no contiene contraseñas ni tokens legibles	`SeguridadTest.S06`	Matriz, fila RNF-SEG-01
RF-14	(Ética: rendición de cuentas) Toda captura, consulta de foto o exportación queda registrada y solo la lee la Dirección	Otros roles reciben 403 al leer el registro	`RF14RegistroActividadTest`	Matriz, fila RF-14
RNF-SEG-07	(Ética: integridad del expediente) Nadie puede fechar un registro en otro día cambiando la hora del teléfono	Fechas de 2001 o 2099 enviadas por el cliente se reemplazan por la del servidor	`SEGRelojServidorTest`	Matriz, fila RNF-SEG-07

9.2. Aviso de privacidad

El aviso integral vive en la app (Perfil → *Acerca de y aviso de privacidad*) y en el repositorio, en `app/src/main/assets/aviso-privacidad.md`. Los dos son el mismo archivo. Se transcribe a continuación.

- **Ley y artículos a los que da cumplimiento:** Ley Federal de Protección de Datos Personales en Posesión de los Particulares, publicada en el DOF el 20 de marzo de 2025 y vigente desde el 21 de marzo de 2025: art. 2 fr. VI (datos sensibles), art. 7 (consentimiento y su revocación), art. 8 (consentimiento expreso y por escrito para datos sensibles), arts. 14–17 (aviso de privacidad, su contenido mínimo y modalidades), art. 18 (medidas de seguridad), art. 21 (representante legal), arts. 22–26 (derechos ARCO), arts. 28, 31 y 34 (solicitud, plazos de 20 y 15 días hábiles y gratuidad), art. 36 (transferencias sin consentimiento) y arts. 40 y 42 (queja ante la Secretaría Anticorrupción y Buen Gobierno).
- **Normas complementarias para menores:** Reglamento de la LFPDPPP (DOF 21-12-2011, art. 89); Código Civil Federal, arts. 23 y 425 (representación por quien ejerce la patria potestad); LGDNNA, arts. 2 y 76 (interés superior y derecho a la intimidad).
- **Fuente:** texto vigente de la Cámara de Diputados, https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf.

Responsable. La Fundación Estancias Infantiles Sonrisas de Cristal, en Monterrey, Nuevo León. Cada estancia tiene una Dirección que atiende las solicitudes de privacidad de sus familias y de su personal (art. 29).

Datos que recopilamos.

- De niñas y niños: nombre, apellidos, fecha de nacimiento y sala. Datos de salud, que son sensibles: alergias, padecimientos, medicamentos, lesiones o síntomas al llegar e incidentes. Su día en la estancia: llegada, salida, comidas, siestas, baño, ánimo y notas. Fotografías, solo con la autorización de imagen para el uso autorizado.
- De tutores y personas autorizadas: nombre, parentesco y teléfono; quién dejó y quién recogió al niño; mensajes enviados a la estancia.
- Del personal: nombre, usuario, rol, estancia y sala.
- De todas las cuentas: usuario, una huella irreversible de la contraseña (PBKDF2) y un registro de actividad con la hora del servidor.

No recopilamos ubicación. A las fotos se les borran los metadatos (EXIF/GPS). Tu huella digital nunca sale del teléfono.

Para qué los usamos.

1. Registrar la asistencia y entregar al niño solo a personas autorizadas.
2. Llevar la bitácora diaria y avisar a la familia de incidentes y de temas de salud.
3. La comunicación entre la familia y la estancia.

4. Compartir con cada familia las fotos donde aparece su hijo.
5. Reportes de asistencia; a la Fundación solo le llegan totales.
6. Proteger las cuentas y registrar quién hizo qué.

No los usamos con fines comerciales ni para decisiones automatizadas.

Consentimiento. Quien ejerce la patria potestad da su consentimiento expreso y por escrito para los datos de salud al inscribir al niño (art. 8). La autorización de imagen se registra por separado en el expediente.

Dónde se guardan.

- En la base PostgreSQL de la Fundación, en un contenedor aislado.
- Viajan cifrados con HTTPS (TLS).
- En el teléfono, la sesión se guarda cifrada con AES-256-GCM, con una llave del Android Keystore.
- Cada persona ve solo lo que le corresponde.

Con quién se comparten. Con nadie fuera de la Fundación, salvo los casos del art. 36 (por ejemplo, un requerimiento de una autoridad competente o una emergencia médica de un niño).

Cuánto tiempo se conservan.

- El expediente, mientras el niño esté inscrito. Al darlo de baja, se borra automáticamente a los 30 días.
- Las fotos, hasta que se borren. De las fotos de grupo, al eliminar un expediente se quita la etiqueta del niño.
- Las cuentas de familia, hasta que se elimina su último expediente, junto con sus conversaciones.
- Las sesiones, 30 días.
- El registro de actividad, mientras opere la estancia. Sus entradas ligadas a un niño se borran con el expediente.

Derechos ARCO. Tú, o quien ejerza la patria potestad del menor (art. 21; art. 89 del Reglamento), puedes pedir a la Dirección, por escrito, el acceso, la rectificación, la cancelación o la oposición. Respondemos en 20 días hábiles y aplicamos lo procedente en los 15 siguientes (art. 31). El trámite es gratuito (art. 34). Si no estás de acuerdo con la respuesta, puedes acudir a la Secretaría Anticorrupción y Buen Gobierno (arts. 40 y 42).

Cambios. Se publican en la app y se anuncian en Avisos (art. 15 fr. VI).

Correspondencia con el comportamiento real.

- **Acceso:** la familia ve todo lo registrado de su hijo.
- **Rectificación:** la Dirección edita el expediente.
- **Cancelación:** baja seguida de eliminación, comprobada por `ARCOEliminacionTest`.
- **Oposición a fotos:** la Dirección cambia la autorización de imagen, y `POST /fotos` la hace cumplir.
- La limpieza de metadatos y el cifrado tienen sus propias pruebas (`SEGArchivosTest`, `S06`).

9.3. Clasificación de los datos

Niveles de sensibilidad:

- **S** (sensible, art. 2 fr. VI): su mal uso puede causar discriminación o un riesgo grave.
- **A** (alto): datos de menores o que permiten localizarlos o identificarlos.
- **M** (medio): datos de contacto o de cuenta.
- **B** (bajo): datos operativos.

Todo dato del servidor viaja por TLS, vive en PostgreSQL dentro de un volumen no expuesto y se entrega solo según la regla de visibilidad por rol. En la tabla, la columna *Protección* indica lo que se agrega a eso.

Dato almacenado	Nivel	Ubicación	Protección aplicada	Conservación
Nombre y apellidos del niño	A	`ninos.datos`	Visible solo para su sala, su familia y su Dirección	Inscripción + 30 días de baja
Fecha de nacimiento del niño	A	`ninos.datos`	Igual que el nombre	Inscripción + 30 días
Alergias del niño	S	`ninos.datos`	Consentimiento escrito; la Fundación no lo ve	Inscripción + 30 días
Padecimientos y medicamentos	S	`ninos.datos`	Igual que alergias	Inscripción + 30 días

Dato almacenado	Nivel	Ubicación	Protección aplicada	Conservación
Permisos de imagen (interno y redes)	M	`ninos.datos`	Validados en el servidor al publicar fotos	Inscripción + 30 días
Estado del expediente y fecha de eliminación	B	`ninos.datos`	Solo lo cambian `/baja` y `/reactivar`, con la hora del servidor	Inscripción + 30 días
Personas autorizadas: nombre	A	`ninos.datos`	Visible para la sala, la familia y la Dirección	Inscripción + 30 días
Personas autorizadas: parentesco y teléfono	M	`ninos.datos`	Igual	Inscripción + 30 días
Tutor: nombre y parentesco	M	`tutores.datos`	Visible para su estancia; borrado con el último hijo	Mientras tenga un hijo inscrito
Tutor: teléfono	M	`tutores.datos`	Igual	Igual
Hora de llegada y de salida	A	`asistencias.datos`	Fecha del servidor; hora limitada a la del servidor	Inscripción + 30 días
Quién entregó y quién recogió	A	`asistencias.datos`	Solo personas autorizadas del expediente (RF-15)	Inscripción + 30 días
Lesiones, síntomas y medicamento al llegar	S	`asistencias.datos`	Visible solo para la sala, la familia y la Dirección	Inscripción + 30 días
Comidas, siestas, baño y ánimo	A	`eventos.datos`	Firmados por la educadora con la hora del servidor	Inscripción + 30 días
Notas de la educadora	A	`eventos.datos`	Igual	Inscripción + 30 días
Incidente: tipo y parte del cuerpo	S	`incidentes.datos`	Folio único; hora del servidor	Inscripción + 30 días
Incidente: descripción y acción tomada	S	`incidentes.datos`	Igual	Inscripción + 30 días

Dato almacenado	Nivel	Ubicación	Protección aplicada	Conservación
Imagen de la foto	A	`archivos.bytes`	Magic bytes, recodificada sin EXIF ni GPS; descarga solo por quien ve la foto	Hasta que se borre
Descripción y niños etiquetados de la foto	A	`fotos.datos`	Visible solo para las familias etiquetadas	Hasta que se borre
Texto del mensaje	M	`mensajes.datos`	Solo participantes; no se copia al registro de actividad	Mientras exista la cuenta
Autor y hora del mensaje	M	`mensajes.datos`	Hora del servidor	Igual
Lecturas de mensajes y avisos	B	`lecturas`, `avisos_leidos`	Por usuario	Mientras exista la cuenta
Quién reclamó un objeto perdido	M	`objetos.datos`	Una familia solo ve su propio reclamo	Mientras opere la estancia
Foto del objeto perdido	B	`archivos.bytes`	Sin metadatos; visible solo para la sala	Mientras opere la estancia
Usuario (login)	M	`usuarios.datos`	No visible para familias	Mientras exista la cuenta
Nombre y rol de la cuenta	M	`usuarios.datos`	Por estancia	Mientras exista la cuenta
Contraseña	S	`claves.hash`	PBKDF2-SHA256, 210 000 iteraciones, con sal; nunca en claro	Mientras exista la cuenta
Token de sesión (servidor)	S	`sesiones.token_hash`	Solo su SHA-256	30 días; los vencidos se purgan
Token de sesión (teléfono)	S	SharedPreferences de la app	AES-256-GCM con una llave del Android Keystore; `allowBackup=false`	Hasta cerrar sesión

Dato almacenado	Nivel	Ubicación	Protección aplicada	Conservación
Desbloqueo con huella activo	B	`huellas`	Solo un indicador; la huella no sale del teléfono	Mientras exista la cuenta
Bloqueo de cuenta	B	`bloqueos`	Solo la Fundación lo ve y lo quita	3 días o hasta el desbloqueo
IP de intentos fallidos	M	Memoria del servidor	No se persiste	24 horas
Registro de actividad	M	`registros.datos`	Solo lo lee la Dirección; hora del servidor	Mientras opere la estancia; lo ligado a un niño, con su expediente

9.4. Dilemas éticos y decisiones de diseño

Dilema 1: compartir fotos o proteger la imagen de los niños.

El dilema. Las fotos tranquilizan a las familias, pero cada foto de grupo expone a otros niños cuyas familias quizá no la autorizaron. Además, el archivo puede llevar la ubicación de la estancia en sus metadatos.

Alternativas y argumentos.

1. Compartir todas las fotos con toda la sala: máxima transparencia y menos trabajo para la educadora.
2. No usar fotos: cero riesgo, pero se pierde un beneficio que las familias valoran.
3. Compartir con reglas: cada foto la ven solo las familias de los niños que aparecen, con permiso de imagen por uso, y cualquiera de esas familias puede borrarla. Es más complejo, pero respeta a cada familia.

Postura del equipo. La alternativa 3, por el interés superior de la niñez (LGDNNA arts. 2 y 76).

Decisión concreta.

- Dos permisos de imagen por niño (RF-16), validados en el servidor.
- La visibilidad de cada foto depende de los niños etiquetados (RF-09).
- Cualquier familia etiquetada puede borrar la foto para todos (RF-10).
- Toda imagen se guarda sin EXIF ni GPS.

- Las fotos se toman dentro de la app y no pasan por la galería del teléfono personal.

Dónde se observa. Pantalla *Fotos* (los niños sin permiso no se pueden etiquetar), `Rutas Contenido.kt`, `RF09FotosVisiblesTest`, `RF10EliminarFotoTest` y `SEGArchivosTest`.

Dilema 2: trazabilidad o privacidad del registro de actividad.

El dilema. Registrar quién hizo qué protege a los niños y permite aclarar incidentes, pero el registro mismo es un expediente sobre el personal y las familias, y puede volverse vigilancia.

Alternativas y argumentos.
1. Registrar todo, mensajes incluidos, y que todos lo consulten: máxima transparencia.
2. No registrar nada: máxima privacidad, pero sin rendición de cuentas.
3. Registrar solo acciones sobre datos de niños (capturas, consultas de fotos, exportaciones, cuentas) y que solo lo lea la Dirección.

Postura del equipo. La alternativa 3: rendición de cuentas proporcional (LFPDPPP art. 5, proporcionalidad).

Decisión concreta.
- RF-14: la lectura del registro está restringida a la Dirección.
- El contenido de los mensajes nunca se copia al registro.
- La consulta de una foto por una familia sí queda registrada.
- Al eliminar un expediente se borran sus entradas del registro.

Dónde se observa. *Más* → *Registro de actividad* (solo Dirección), `RF14RegistroActividadTest` y `ARCOEliminacionTest`.

Dilema 3: borrar de verdad o conservar evidencia.

El dilema. El derecho de cancelación pide borrar los datos de un niño que deja la estancia. Pero borrar al instante puede destruir evidencia de un incidente, o un error de la Dirección puede borrar un expediente activo.

Alternativas y argumentos.
1. Borrado inmediato: cumple la cancelación al momento, pero un error es irreversible.
2. Nunca borrar (solo desactivar): conserva la evidencia, pero contradice la ley y el aviso de privacidad.
3. Baja con un periodo de gracia de 30 días y luego borrado completo: permite corregir errores y luego cumple la cancelación de punta a punta.

Postura del equipo. La alternativa 3.

Decisión concreta.
- El servidor rechaza eliminar un expediente activo (S20).
- La purga automática corre cada 6 horas.
- El borrado incluye las imágenes, las entradas del registro, la cuenta de la familia y sus chats (`ARCOEliminacionTest`).
- En la auditoría se corrigió que las imágenes quedaban huérfanas.

Dónde se observa. Expediente → *Dar de baja* y *Eliminar definitivamente*; sección 7 del aviso de privacidad.

10. Entregables y enlaces

- **Repositorio:** github.com/santihdzs/sonrisas-de-cristal; versión entregada en el tag `v3.0-entrega-final`.
- **APK de release firmado (3.0.0):** `entrega/sonrisas-de-cristal-v3.0.0.apk`.
- **Instrucciones para levantar el backend:** sección 6 (Manual de instalación y ejecución) y `README.md` en GitHub.
- **Aviso de privacidad:** `app/src/main/assets/aviso-privacidad.md`, también dentro de la app.
- **Video de respaldo de la demostración:** video en Google Drive.
- **Plan de Planner:** tablero NeuralHub en Microsoft Teams; captura del tablero final en la sección de administración.
- **Presentaciones:** `docs/presentaciones/presentacion-1.pdf` y `docs/presentaciones/presentacion-2.pdf`.

A. Archivos de evidencia

Rutas relativas a la raíz del repositorio; cada una enlaza al archivo en GitHub en el tag entregado.

Archivo	Contenido
`docs/seguridad/zap/reporte-zap.html`	Reporte de OWASP ZAP del escaneo autenticado (también en `.json` y `.md`)

Archivo	Contenido
docs/seguridad/zap/antes-reporte-zap.html	Escaneo previo, sin sesión, con la alerta baja que se corrigió
docs/seguridad/zap/hallazgos.md	Justificación o mitigación de cada alerta y códigos de respuesta durante el escaneo
docs/seguridad/zap/openapi.json	Especificación OpenAPI usada por ZAP, generada desde las rutas del backend
docs/seguridad/verificacion-despliegue.txt	Cabeceras por el túnel HTTPS, login de cada rol, dato capturado por la educadora y leído por la familia, `docker inspect`
docs/pruebas/resultados.txt	Pruebas del backend por clase, en H2 y en PostgreSQL 17
docs/pruebas/reporte-backend/index.html	Reporte HTML de JUnit
backend/src/test/kotlin/.../servidor/	Código de las pruebas citadas en la matriz
evidencia-pruebas/	Capturas por rol y matriz de la Entrega 2
docs/presentaciones/presentacion-1.pdf	Presentación 1
docs/presentaciones/presentacion-2.pdf	Presentación 2 (final)